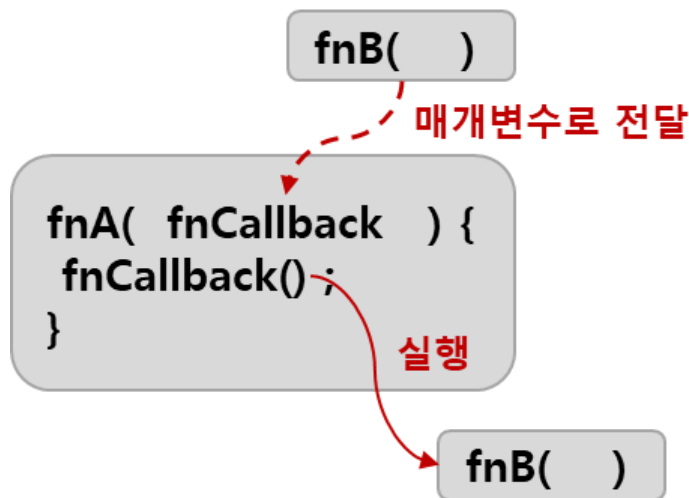


1. 콜백 함수

자바스크립트는 일급 함수로 일급함수의 특성을 사용해서 비동기 처리 방식을 구현하기 위해 콜백 함수(CallBack Function)을 사용합니다.

일급함수란 프로그래밍 언어가 함수를 first-class citizen으로 취급하는 것을 뜻하는 것으로 수함 자체를 인자로써 다른 함수에 전달하거나 다른 함수의 결과 값으로 리턴할 수도 있고, 함수를 변수에 할당하거나 데이터 구조 안에 저장할 수 있는 함수를 뜻합니다.

- 콜백은 함수이며 호출 되는 시점에 제어권이 넘어 간다.
- 콜백 지옥은 콜백 함수를 익명 함수로 전달 하는 과정에서 반복되어 코드가 복잡 해 지는 것을 의미 한다.



callback 함수

```
{% code title="index.html" lineNumbers="true" %}
```

```
<!DOCTYPE html>
<html lang="kr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML</title>
</head>
<body>
  <Label id="msg00" ></Label></br>
  <Label id="msg01" ></Label></br>
  <Label id="msg02" ></Label></br>
</body>
<script>
  // 콜백 함수
  const fnB = (name) => {
    console.log(`fnB :: ${name}`);
    document.querySelector("#msg00")
      .innerHTML = `msg00 :: ${name} :: 시간 :: ${new
Date().getTime()}`;
  };

```

```

const fnA = (name, fnCallback) => {
  setTimeout(() => {
    fnCallback(name);
  }, 1000);
  console.log(`fnA :: ${name} , 함수 ${fnCallback}`);

  document.querySelector("#msg01")
    .innerHTML = `msg01 ::  ${name} :: 시간 :: ${new
Date().getTime()}`;
};

// 매개 변수로 함수 생성해서 전달
fnA('김길동', function fnZ(pname) {
  document.querySelector("#msg02")
    .innerHTML = `msg02 :: ${pname} :: 시간 :: ${new
Date().getTime()}`;
});

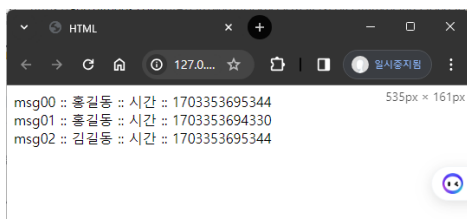
// 매개 변수로 함수 전달
fnA('홍길동', fnB);

</script>
</html>

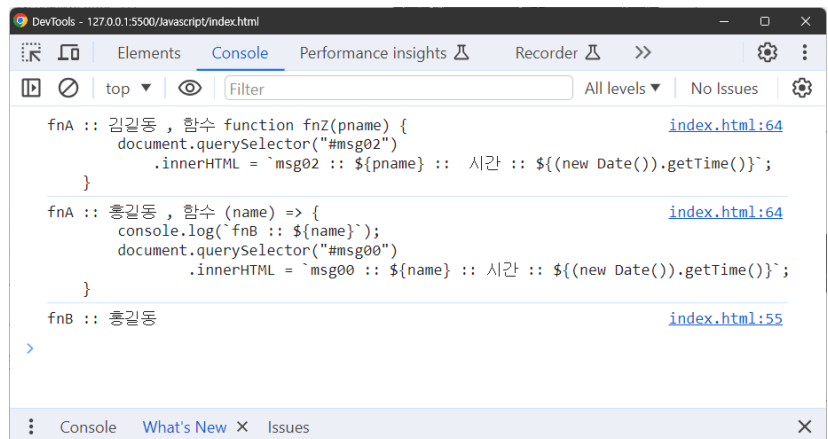
```

{% endcode %}

실행 결과



실행 콘솔



실행 순서

- 32 line 실행 : 콜백 함수를 생성해서 매개변수로 넘김
 - 22 line 에서 1초 후 매개변수로 넘긴 함수 실행
- 25 line 실행 : 콘솔에 김길동 출력
- 27 line 실행 : 화면에 김길동 출력 (Label : msg01)
- 38 line 실행 : 콜백 함수로 fnB를 매개변수로 넘김
 - 22 line 에서 1초 후 매개변수로 넘긴 함수 실행 (fnB)
- 25 line 실행 : 콘솔에 홍길동 출력
- 27 line 실행 : 화면에 홍길동 출력 (Label : msg01)

7. 1번에서 콜백 함수 실행 : 화면에 김길동 출력 (Label : msg02)
8. 4번에서 콜백 함수 실행 : 화면에 홍길동 출력 (Label : msg00)

1. 콜백은 함수

```
var cnt = 0;
var fTimerCB = function() {
  if (cnt == 0 ) console.log("cnt 0");
  console.log(cnt);
  if (++cnt > 4) {
    clearInterval(timer);
    console.log("cnt ::" + cnt);
  }
};

var timer = setInterval(fTimerCB, 300);
console.log("start");

var ObjA = {
  values: [1,2,3],
  fPrintValues: function(value, index) {
    console.log(this, value, index);
  }
}

ObjA.fPrintValues(10,10);
// { values:[ 1, 2, 3 ],fPrintValues:[Function: fPrintValues] } 10 10
[10,20,30].forEach(ObjA.fPrintValues);
// global ,10 ,0 // global,20,1 // global
```

2. 콜백 함수 단점

- 업무 로직이 복잡하다면 콜백에서 콜백... 즉 콜백 함수가 늘어나서 관리가 힘들고 복잡해져 유지 보수가 힘든 상태로 빠질수 있다. 흔히 콜백 지옥(callback hell)이라 함
- 콜백 함수가 실행 했던 시점과 콜백 함수가 종료 된 후 반환값을 처리하는 시점이 분리되어 있어 관리가 어렵고 콜백 1에서 반환된 결과와 콜백2에서 반환된 값에 대한 처리 로직이 분리 되어 있어 오류 발생시 원인 찾기가 힘듭니다.
- 가장 큰 문제점 : 가독성
 - 함수가 길어 지면 시작과 끝을 알 수 없음
 - 각 스코프에서 사용하는 변수 중복 가능
- 클로저가 성능 및 메모리 소비 측면에서 문제 발생
 - 가비지 수집 시 유지 됨

3. 콜백 지옥

setTimeout을 사용한 콜백 지옥 예제 입니다. 업무로직이 복잡 하면 소스를 읽기 어렵고 유지보수가 어려워집니다.

많은 클로저와 in-place 콜백 정의가 코드의 가독성을 떨어뜨리고 관리할 수 없는 코드 (pyramid of doom)

```

var car = '';
setTimeout(function (name) {
  // 업무 로직 1
  car += name + ' ';
  console.log(name); // 소나타
  setTimeout(function (name) {
    // 업무 로직 2
    car += name + ' ';
    console.log(name); // SM5
    setTimeout(function (name) {
      // 업무 로직 3
      car += name + ' ';
      console.log(name); // 그랜저
      console.log('Last : ' + car); // Last : 소나타 SM5 그랜저
    }, 100, '그랜저');
  }, 100, 'SM5');
}, 100, '소나타');

```

- 이 문제를 해결하는 방법은 함수로 분리 하는 것 입니다. (기명 함수 변경)
익명 함수로 기명 함수로 변경 하여 가독성 높임 순차적으로 실행 하게 변경

```

var car = '';
var car1 = function (name) {
  car += name + ' ';
  console.log(name);
  setTimeout(car2, 100, 'SM5');
};

var car2 = function (name) {
  car += name + ' ';
  console.log(name);
  setTimeout(car3, 100, '그랜저');
};

var car3 = function (name) {
  car += name + ' ';
  console.log(name);
  console.log('Last : ' + car);
};

setTimeout(car1, 100, '소나타');

```

함수로 분리해서 각 함수에 대한 코드 가독성은 좋아지나 업무 흐름을 파악하기 위해서는 call flow를 확인해야 합니다